Local multi-agent self-play RL for software engineering

Libor "Burny" Burian

Run: pilot-recovery | Status: not_evaluated

Abstract

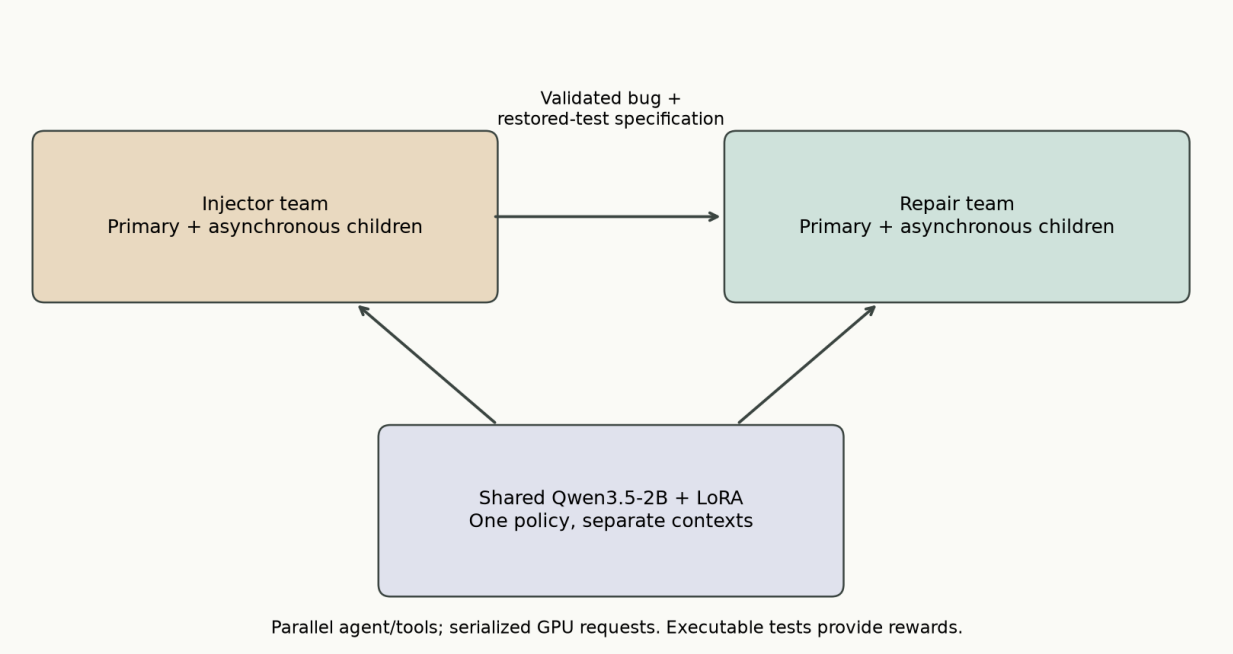
We implement a local shared-policy extension of Self-play SWE-RL in which bug-injection and repair teams can delegate to asynchronous subagents. All roles use one Qwen3.5-2B policy with a shared trainable LoRA adapter. This report is generated from the saved run records. Recorded run status: not_evaluated. No completed policy optimizer steps are recorded; this record does not establish a learned checkpoint. Not evaluated: no tasks have completed outcomes for both conditions.

Context and motivation

The experiment asks whether reinforcement learning from executable repository feedback improves held-out human-bug repair. Prior SSR work motivates repository-grounded bug injection and repair [1]. Anchored Self-Play demonstrates that gains on synthetic bugs need not transfer to human bugs [7], so the two evaluation sources remain separate.

Method

The injector edits source and weakens existing tests. A trusted Python runner validates the artifact, including restoration checks and inverse mutations. The repair team sees the weakened buggy state and the inverse weakening patch, but not clean reference code or the injection patch. Original oracle tests are restored for scoring. This Python test interface is a deliberate adaptation of SSR's more general artifact-discovery procedure [1].



Task construction and trusted validation

The clean selected test suite must contain at least five passing tests. The source mutation must induce a failure and preserve a pass. The weakening patch must hide at least one induced failure. Inverse mutation checks revert each changed source file independently and require that reversal to repair at least one induced failure. Empty

selections, missing expected tests, collection failures, and parser errors cannot produce success. Artifacts retain the SSR five-file interface: source mutation, test weakening, selected test files, runner, and parser; the last two are constructed by this harness rather than delegated to a model [1]. One failed repair per original bug can become an independently revalidated second-order task. Its repair attempts contribute solver gradients; the original injector return uses first-order attempts only.

Agents and optimization

Primary agents delegate through a tool patterned after OpenCode [3]. Child sessions have independent contexts and can work while the primary agent continues. GPU generation requests are serialized through the shared policy. One team execution includes all descendant actions. Each actual generated token contributes once; a child's text repeated in its parent's context is an observation. The update extends CWM's length-weighted centered returns, clipped token likelihood ratios, and action masks [2]. The precise run configuration is included in the appendix.

Exact objective and team credit

For a group G of episodes with the same task and team role, let L_e be the number of unique generated action tokens and R_e the episode return. The length-weighted baseline and advantage are mu_G = sum_e(L_e R_e) / sum_e(L_e), and A_e = R_e - mu_G. Injector and solver groups remain separate. Every primary/child action in episode e receives A_e; prompt tokens, tool output, and copied child output receive no action loss [2].

For an action token t, r_et = exp(log pi_theta(a_et | original_context_et) - old_logprob_et). The minimized objective is L(theta) = -(1/C) sum_G sum_e sum_t min(r_et A_e, clip(r_et, 0.8, 1.25) A_e). C is the maximum planned collection token capacity, not the observed trajectory length. In primary self-play, C = injector_samples * (1 + repair_samples * max_bug_order) * team_generated_token_budget. There is no reward-standard-deviation normalization or KL term. Each generated call appears once in the collection and once per optimization pass. The original behavior log probabilities remain fixed across both passes [2].

Injector returns follow SSR: -1 for an invalid artifact; -0.8 for a valid bug with empirical repair success rate s equal to zero or one; otherwise 1 - 1.8s. A repair receives +1 only when the restored oracle passes completely, and -1 otherwise [1]. Zero-advantage groups are recorded without an update. Incomplete or infrastructure-corrupted groups never enter optimization.

Model, sampling, and optimization settings

The recorded model is Qwen/Qwen3.5-2B at revision 15852e8c16360a2fea060d615a32b45270f8a8fc. LoRA rank is 16, alpha 16, and learning rate 5e-06; recorded training seed is 0. The implementation uses BF16 adapters with dropout zero, a frozen base, frozen vision tower, embeddings and output head, and language-model attention/MLP adapters only [4, 5].

A call has at most 2048 input-plus-action tokens and 256 generated tokens. Each team shares a generated-token cap of 4096, a tool-call cap of 32, and 2 child sessions at depth 1. Sampling temperature is one; thinking is disabled. Root and child threads progress concurrently, but calls acquire one shared GPU policy lock. All workers finish before an update.

The recorded collection uses 2 independent injectors per clean starting state and 4 repairs per valid bug, followed by 2 optimization passes. AdamW uses betas (0.9, 0.99), weight decay zero, gradient clipping at norm 1.0, gradient checkpointing, and one call per microbatch. Likelihood computation retains only action-position logits and accumulates gradients sequentially.

Evaluation and analysis

Frozen and final policies are compared on matched tasks and inference budgets. Task-level paired percentile bootstrap intervals use the resampling scheme in [8]. Only completed boolean outcomes are scored.

Infrastructure failures and unfinished evaluations are reported as missing coverage, not failed repairs. Conflicting duplicate completed outcomes are rejected. The final checkpoint is selected by completion order rather than held-out score.

Evaluation sampling and stopping

Planned human and synthetic task counts in the run manifest are 32 and 16. Human tasks are fixed before training by sorting SHA-256 instance-ID hashes within the eligible requests, Flask, pylint, and pytest repositories. Synthetic humanize mutations are selected by deterministic syntax and executable validity, with at most one accepted mutation per function. Neither cohort is selected using model performance. Gold human fixes and evaluator test patches are withheld from agents [6].

Each source is analyzed separately with 10000 task resamples, bootstrap seed 0, and percentile endpoints at 2.5% and 97.5%. The paired difference is mean(trained_success - base_success) over completed pairs. A positive lower bound supports the prediction; an upper bound at or below zero contradicts it; an interval spanning zero is inconclusive. Degenerate intervals are explicitly qualified [8].

This run's recorded single-GPU wall-clock cap is 28200 seconds. It includes model loading, compilation, preflight, baseline, training, and final evaluation, using a persisted deadline that restarts cannot reset. Baseline allocation is capped at (remaining - 1800) / 2.25 seconds. Training reserves the greater of the configured minimum final-evaluation reserve and 1.25 times measured baseline runtime. A new collection must fit its measured duration with margin. If an optimizer pass crosses the reserve boundary, restore the last complete checkpoint. Nonfinite gradients, memory failure, likelihood incompatibility, or corrupt state stop training without a silent model/precision substitution. Preregistration and any amendments remain authoritative [9].

Differences from prior SSR

This is a Python-only, local, single-GPU adaptation, not a numerical reproduction of SSR [1]. It uses Qwen3.5-2B, one tied LoRA adapter for both teams and all descendants, asynchronous prompted delegation, trusted pytest execution/JUnit parsing, bounded contexts, and a small fixed evaluation cohort. It does not reproduce SSR's original model scale, context length, distributed asynchronous learner, git-history-aware bug-injection prompts, or full evaluation suite. The CWM objective is extended from trajectories to complete action trees using the exact original contexts [2].

Interrupted attempt and recovery amendment

Prior attempt runs/pilot has recorded status preflight_failed. Its original wall-clock deadline was 2026-10-30T15:31:24.450111+00:00; original deadline preserved: True. Recorded prior failure: preflight: BrokenPipeError: [Errno 32] Broken pipe. The prior attempt's recorded elapsed time was 479.063 seconds. The recovery amendment charges 600 seconds for that attempt and allocates 28200 seconds to this recovery, within 28800 seconds of total active allocation. The amendment excludes the offline interval; it does not erase the expired original attempt. Saved prior artifacts: model outputs 0, evaluations 0, checkpoints 0. Recorded reason: User requested recovery after OS restart. Original preflight failed with BrokenPipeError; original wall-clock deadline expired. Offline interval is excluded only by the dated preregistration amendment.

Results

Recorded run status: not_evaluated. No completed policy optimizer steps are recorded; this record does not establish a learned checkpoint.

Not evaluated: no tasks have completed outcomes for both conditions.

Synthetic diagnostic set: Not evaluated: no tasks have completed outcomes for both conditions.

Source	Policy	Planned	Attempted	Resolved / scored	Rate	Unscored recorded	Unrecorded
human	base	32	0	Not evaluated	Not evaluated	0	32
human	trained	32	0	Not evaluated	Not evaluated	0	32
synthetic	base	16	0	Not evaluated	Not evaluated	0	16
synthetic	trained	16	0	Not evaluated	Not evaluated	0	16

Deferred controls (not executed in this pilot)

single-agent self-play: not run. Separate delegation from self-play training.

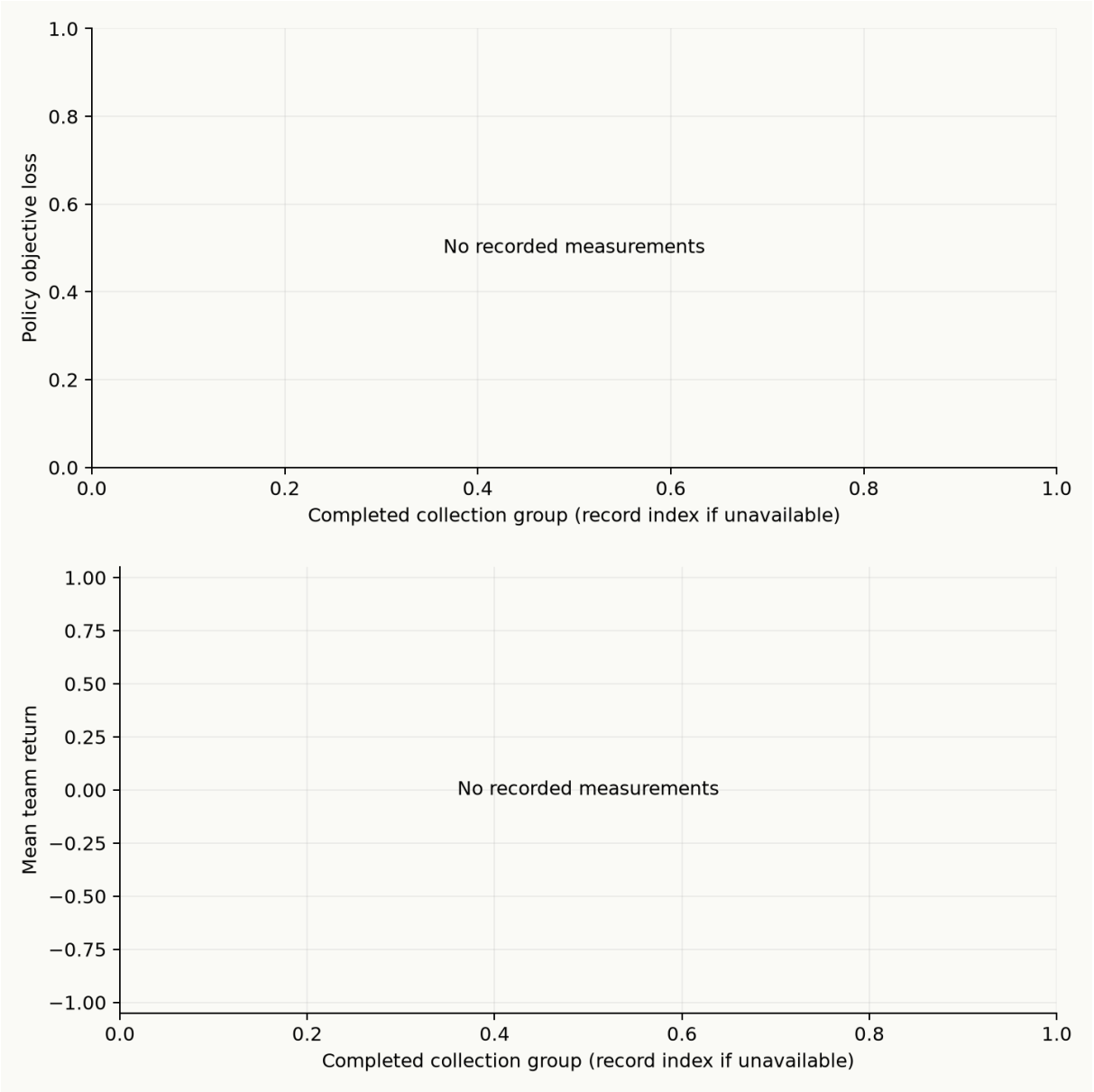
repair-only training: not run. Control for updating the injector.

procedural-mutation training: not run. Control the training bug distribution.

no-delegation evaluation: not run. Measure inference-time delegation dependence.

seed sensitivity: not run. Measure training-seed variation.





Observed execution diagnostics

The following diagnostics are derived from saved event, evaluation, and optimizer streams, including incomplete collections. Counts describe recorded activity, not generalization. Preflight memory belongs to a software check, not a trained-model evaluation. Recorded action tokens: 0; actual subagent tool calls: 0; child sessions: 0; optimizer steps: 0. Full invalidity reasons, exclusions, failure metadata, and per-task outcomes accompany the generated report.

episodes: 0

child sessions: 0

episodes with children: 0

delegation rate: Not measured

delegation calls: 0

generated action tokens: 0

generation calls: 0

validated artifacts: 0

invalid artifacts: 0

invalid artifact rate: Not measured

invalid reasons: {}

untrained group reasons: {}

optimizer steps: 0

recorded training groups: 0

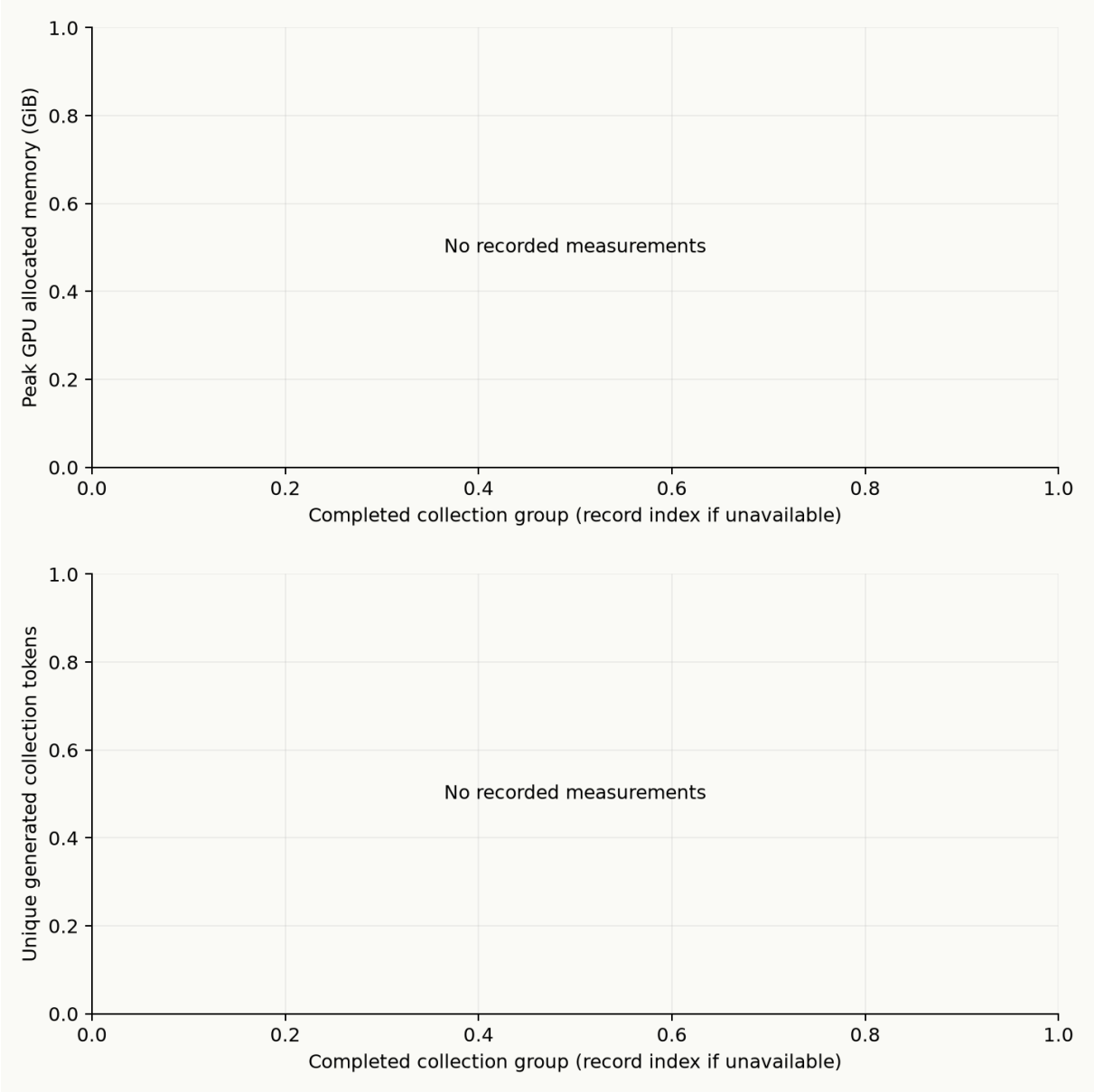
peak allocated bytes: Not measured

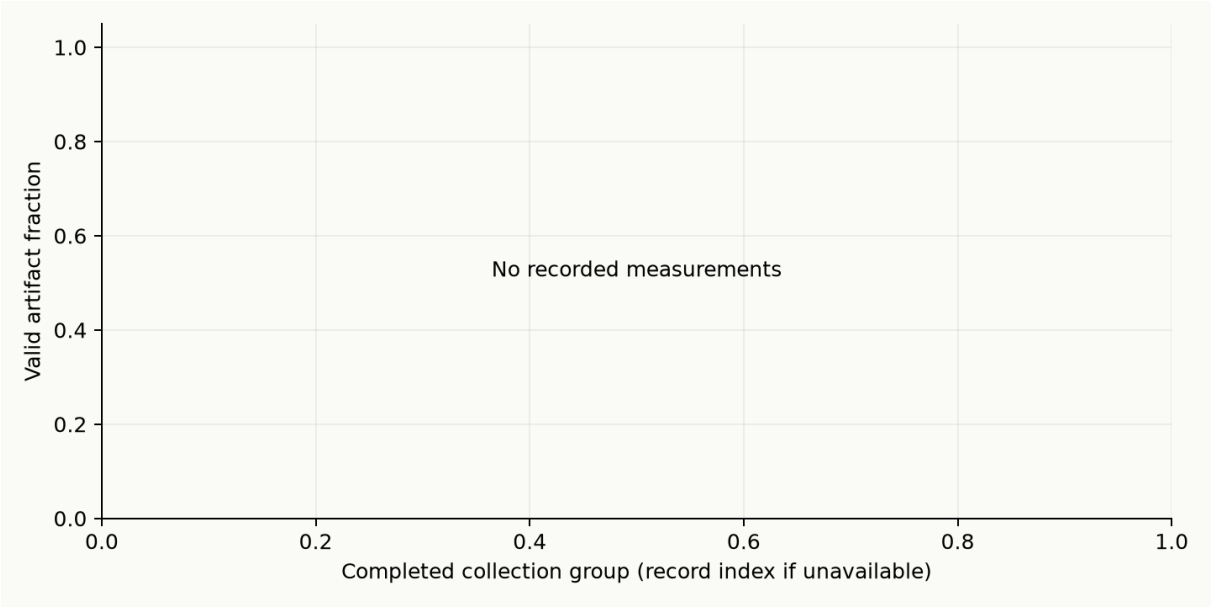
evaluation seconds: 0

training group seconds: 0

pass to pass regressions: Not measured

regression scored records: 0





Claims and implications

These measurements are a comparison of the recorded policies under this harness. They do not isolate the causal contribution of delegation, self-play, or parameter sharing. Those claims require the planned controlled comparisons. A functioning update or improved training reward alone does not establish better software engineering.

Related work

SSR supplies the bug/repair self-play task [1], CWM supplies the optimization foundation [2], and OpenCode supplies a reference for tool-mediated child sessions [3]. The model and local LoRA configuration follow Qwen and Unsloth documentation [4, 5]. Human-bug scoring uses the SWE-bench harness [6]. This implementation does not claim the first bug/repair self-play method.

Limitations and discussion

The pilot uses one training seed, a small task sample, a short context window, and limited compute. Task bootstrap intervals condition on this checkpoint and cannot quantify training-seed variation. Small samples can yield degenerate intervals. Public repositories may have appeared in model pretraining; contamination is unknown. Trusted tests reduce particular scoring failures but cannot guarantee realism or prevent every form of reward gaming. Unrun controls are not evidence. All incomplete coverage is retained in the tables.

Conclusion

The saved artifacts establish what ran and what was measured. Not evaluated: no tasks have completed outcomes for both conditions.

Appendix: recorded configuration

code_commit: 57f5624bd567110799ee098a7c0ea27d5abef608

config: {"model": {"model_id": "Qwen/Qwen3.5-2B", "revision": "", "context_tokens": 2048, "action_tokens": 256, "lora_rank": 16, "lora_alpha": 16, "learning_rate": 5e-06, "seed": 0}, "limits": {"generated_tokens": 4096, "tool_calls": 32, "child_sessions": 2, "child_depth": 1, "command_seconds": 90, "episode_seconds": 900}, "experiment": {"budget_seconds": 28200, "injector_samples": 2, "repair_samples": 4, "optimization_passes": 2, "max_bug_order": 2, "human_tasks": 32, "synthetic_tasks": 16, "minimum_final_reserve_seconds": 7200, "delegation": true, "condition": "selfplay", "bootstrap_samples": 10000, "recovery_evidence":

"manifests/recovery.json"}}

created_at: 2026-11-05T20:58:47.386714+00:00

expected_human_tasks: 32

expected_synthetic_tasks: 16

hardware: name, memory.total [MiB], compute_cap NVIDIA GeForce RTX 5070 Laptop GPU, 8151 MiB, 12.0

hypothesis: Trained team improves held-out human bug resolution over frozen initialization

model: {"model_id": "Qwen/Qwen3.5-2B", "revision": "15852e8c16360a2fea060d615a32b45270f8a8fc", "context_tokens": 2048, "action_tokens": 256, "lora_rank": 16, "lora_alpha": 16, "learning_rate": 5e-06, "seed": 0}

recovery: {"prior_run": "runs/pilot", "prior_status": "preflight_failed", "prior_budget": {"started_epoch": 1793345484.450111, "budget_seconds": 28800, "deadline_epoch": 1793374284.450111, "started_at": "2026-10-30T07:31:24.450125+00:00"}, "prior_events": [{"timestamp": "2026-10-30T07:31:26.384084+00:00", "event": "phase_started", "phase": "preflight", "code_commit": "a49d13369ec17c005c844af17b88aa6dff66d86a", "remaining_seconds": 28798.066036701202}, {"timestamp": "2026-10-30T07:39:23.513025+00:00", "event": "phase_failed", "phase": "preflight", "error": "BrokenPipeError: [Errno 32] Broken pipe"}], "prior_attempt_elapsed_seconds": 479.0629141330719, "charged_prior_seconds": 600, "recovery_budget_seconds": 28200, "total_active_allocation_seconds": 28800, "original_deadline_preserved": true, "saved_model_generations": 0, "saved_checkpoints": 0, "saved_evaluations": 0, "original_record_sha256": {"budget.json": "c2efc46ea1aa4ee59e82a08855ddfb74b3bb4811a45b39b625b55d1bed5202e4", "events.jsonl": "0387ecd61f92982dc0ccdd31746f45b8f276885d72b20318c66c45bac00ecc27", "manifest.json": "15d1cfe5cd7cd423c536b23b5e0b60644258ed9e72b99d2c0c1a447d429a0160", "terminal.log": "b529901c78267e59030d5ee1fdd0687a60970ab8b3867a0f40a3cb63200af8fd"}, "reason": "User requested recovery after OS restart. Original preflight failed with BrokenPipeError; original wall-clock deadline expired. Offline interval is excluded only by the dated preregistration amendment."}

References

[1] Self-play SWE-RL: task construction and rewards. https://arxiv.org/html/2512.18552v3

[2] CWM: clipped token objective and trajectory masks. https://arxiv.org/html/2510.02387v1#A3

[3] OpenCode: asynchronous child sessions. https://github.com/anomalyco/opencode/blob/e761e265d2d5e6ff4a1c0756ab8d8eeee0db501e/packages/core/src/tool/plugin/subagent.ts

[4] Qwen3.5-2B model card. https://huggingface.co/Qwen/Qwen3.5-2B

[5] Unsloth Qwen3.5 fine-tuning. https://unsloth.ai/docs/models/qwen3.5/fine-tune.md

[6] SWE-bench evaluation harness. https://www.swebench.com/SWE-bench/guides/evaluation/

[7] Anchored Self-Play: synthetic-to-human bug distribution shift. https://arxiv.org/html/2607.03523v1

[8] Paired percentile bootstrap procedure. https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.bootstrap.html

[9] Committed preregistration and analysis plan. https://github.com/BurnyCoder/multi-agent-llm-self-play-reinforcement-learning-for-software-engineering/blob/main/reports/preregistration.md

AI statement

Codex wrote this codebase and paper from a specification provided by the human author. Human cleanup and rewriting are in progress.